

Homework 4

Recitation

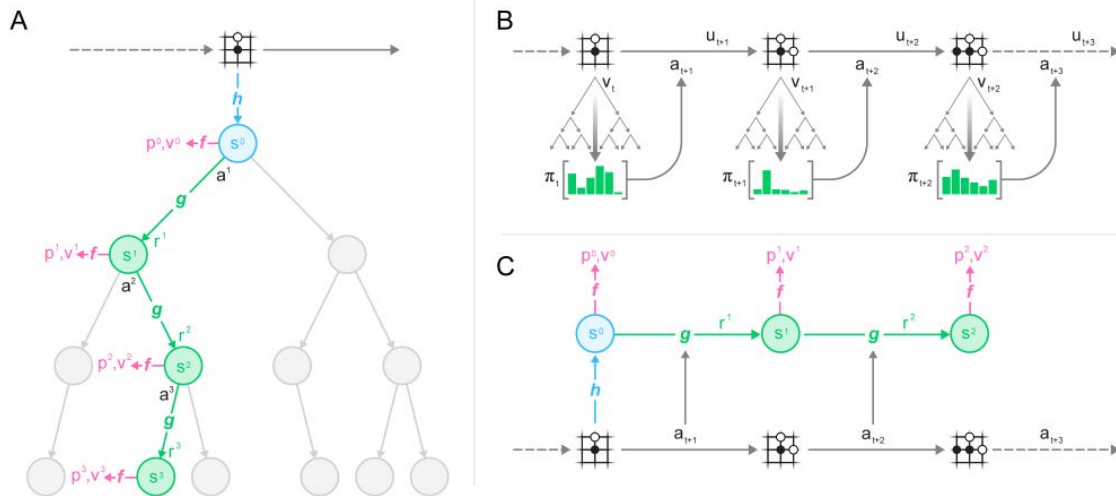


MuZero

- **Building Blocks of AlphaGo, AlphaZero**
- **Based on MCTS, and relaxes assumptions from AlphaGo.**
 - **No game knowledge required.**
 - **Hybrid learned model and policy with UCB-based search.**
 - **Applies to single-player games.**

MuZero Overview

- Search tree is built iteratively, where nodes represent a discrete simulation state, and at every edge we store $\{N(s, a), Q(s, a), P(s, a), R(s, a), S(s, a)\}$



MuZero Overview

At each edge, we store the visit count, q function, policy prior, predicted reward, and next state.

Actions are selected to maximize an upper confidence bound.

We iteratively expand the tree using model predictions.

$$a^k = \arg \max_a \left[Q(s, a) + P(s, a) \cdot \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \left(c_1 + \log \left(\frac{\sum_b N(s, b) + c_2 + 1}{c_2} \right) \right) \right]$$

$$\{N(s, a), Q(s, a), P(s, a), R(s, a), S(s, a)\}$$

MuZero Networks

- Prediction Network (f_{θ})
 - Given Hidden State $\rightarrow$ Return Value Estimate, Policy Priors for the State
- Dynamics Network (g_{θ})
 - Given Hidden State, Action Pair $\rightarrow$ Return Intermediate Reward, Next State Prediction
- Representation Network
 - Given State $\rightarrow$ Return Hidden State Representation

$$r^l, s^l = g_{\theta}(s^{l-1}, a^l)$$

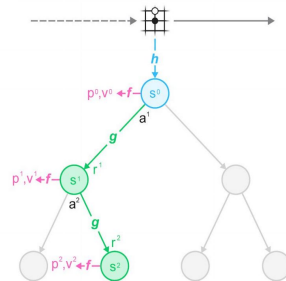
$$\mathbf{p}^l, v^l = f_{\theta}(s^l)$$

Planning with a Learned Model

representation $h_{\theta}(o_1, \dots, o_t) = s^0$

prediction $f_{\theta}(s^k) = \mathbf{p}^k, v^k$

dynamics $g_{\theta}(s^{k-1}, a^k) = r^k, s^k$



Our MuZero Networks

- *representation_network*: This network takes as input the observed state from the environment o and maps it to a latent state representation s .
- *dynamic_network*: This network takes as input a latent state representation s and an action a and predicts the latent state representation of the next state s' .
- *value_network*: This network takes as input a latent state representation s and estimates the corresponding state value v_s .
- *policy_network*: This network takes as input a latent state representation s and maps it to the corresponding policy p_s (use a softmax on the output of the policy network to recover the action distribution).
- *reward_network*: This network takes as input a latent state representation s and an action a and predicts the corresponding reward $r_{s,a}$.

How are our networks trained?

Loss Function Overview:

- *Value Loss*: Cross-Entropy loss between value targets and value outputs from the network's *initial model* or *recurrent model*. We use a cross-entropy loss since the value network outputs a categorical representation of the value functions (this keeps gradients equal regardless of the value size). We also scale the value loss by 0.25. This causes the value loss to be on a similar scale than the reward and policy loss. *Hint*: `tf.nn.softmax_cross_entropy_with_logits` will be useful.
- *Reward Loss*: Mean-Squared Error loss between the predicted rewards and true environment rewards. MuZero does not compute a reward loss for the initial state inference.
- *Policy Loss*: Cross-Entropy loss between policy targets and policy outputs from the network's *initial model* or *recurrent model*.